

Polymorphism, Types

Variable type vs. runtime type

```
Human human = new Human()  
Variable_type name = new Runtime_type()
```

- The **variable type** determines the type of a variable and thereby its possible content
- The **runtime type** designates the type of an object and can be queried via the `.GetType()` method

Polymorphism

- is the "third pillar of OOP" (encapsulation, inheritance, polymorphism)
- Greek for "multiformity"
- objects of derived class can always be considered as objects of parent classes

```
Species s = new Human();
```

Type test with GetType() and typeof operator

- with ".GetType()" the runtime type of an object is determined
- the typeof operator retrieves the System.Type instance for a type
- exact comparison

```
Species s = new Human();
```

```
if (s.GetType() == typeof(Human)) //true
```

```
if (s.GetType() == typeof(Species)) //false
```

Type test with the is operator

- the is operator checks if the runtime type is compatible with a given type
- returns true if the object can also be considered as the requested type / if a derivation exists

```
Species s = new Human();
```

```
if (s is Human) //true  
if (s is Species) //true  
if (s is object) //true
```

virtual members

- the member of the respective instance is always accessed
- the instance can also be cast to the base class, the call is still made to the actual instance
- casting into base class makes sense, if several objects of different derived classes are processed

```
Species s = new Species();  
s.WhatAmI(); //I am a species
```

```
Human h = new Human();  
h.WhatAmI(); //I am a species with two legs and I can speak
```

```
Species hAsSpecies = h as Species;  
hAsSpecies.WhatAmI(); //I am a species with two legs and I can speak
```

Hiding base class members

- Members from the base class can also be hidden
- put the keyword "new" in front of the return value and use the same member name
- if an instance is cast to the base class, the base class can now no longer access the new member

```
public class Species
{
    public virtual string WhatAmI()
    {
        return "I am a species";
    }
}
```

```
public class Human : Species
{
    public new string WhatAmI()
    {
        return "I am a human";
    }
}
```

```
Human h = new Human();
Species hAsSpecies = h as Species;
h.WhatAmI(); //I am a human
hAsSpecies.WhatAmI(); //I am a species
```

abstract members and classes

- are only signatures and specify the structure, but cannot be initialized themselves
- must be overwritten by derived classes
- if a member should be abstract, the class must be marked as abstract

```
public class Vehicle
{
    public abstract void Drive();
}

public class Car : Vehicle
{
    public int MaxV;

    public override void Drive()
    {
        Console.WriteLine("Vehicle is driving");
    }
}
```

abstract members and classes

- cannot use "sealed" as they are mutually exclusive
- a derived class must implement all abstract members
- abstract methods have no body but are only declared